

TASI Intraday Trading System

Operations Procedure & System Reference

Version 1.3 | Generated 20 May 2026 | Confidential

This document describes the full architecture, stakeholder roles, automated workflows, and operational procedures for the TASI (Saudi Stock Exchange) intraday algorithmic trading system running on Ocean server. It covers system components, cron schedules, execution flows, risk controls, and escalation procedures.

Table of Contents

- 1. System Overview & Infrastructure
- 2. Stakeholders & Roles
- 3. System Components
- 4. Cron Schedule & Automation
- 5. Daily Trading Operation Flow
- 6. Execution Flow - Order Lifecycle
- 7. Risk Controls & Stop Logic
- 8. Alert & Escalation Matrix
- 9. Session Health & Keepalive
- 10. Recovery Procedures
- 11. Known Issues & Fixes Log

1. System Overview & Infrastructure

Infrastructure

The trading system runs entirely on Ocean, a dedicated Linux server (Ubuntu 24.04) located at Amin's premises. Ocean connects to Amin-PC (Windows) via a persistent SOCKS5 SSH tunnel for residential IP routing. All browser automation runs headlessly on Ocean's X display.

- Ocean Server - Ubuntu 24.04, 8GB RAM, 240GB SSD - primary compute node
- Amin-PC (Windows) - residential IP proxy via SOCKS5 tunnel on port 1080
- Google Chrome on Ocean - CDP port 18801 - hosts Derayah sessions + TickerChart feed
- Chrome Remote Desktop - PIN 056187 - for visual access to Ocean desktop
- OpenClaw Gateway - port 18789 - AI agent orchestration layer
- Ocean Dashboard - port 8765 - system status web UI
- SOCKS5 Tunnel - autossh to amin-pc, port 1080 - routes Derayah traffic via residential IP

Key Directories

- /home/mino/tasi-exec/ - all trading scripts, logs, and state files
- /home/mino/.openclaw/ - OpenClaw agent config and cron definitions
- /home/mino/.config/google-chrome/derayah-profile/ - Chrome persistent session

2. Stakeholders & Roles

Amin (Owner)	5529987063	Decision maker. Receives all trade alerts, regime reports, and system health notifications via Telegram DM. Can send manual BUY/SELL commands. Final authority on all positions.
Mino (AI Agent)	OpenClaw/Claude	24/7 autonomous agent. Runs all cron jobs, monitors system health, executes keepalive tasks, handles session recovery, and orchestrates the trading pipeline. Communicates via Telegram.
TASI Exec Bot	@TASIExecBot (8989533040619)	Telegram bot running on Ocean (bot.py). Listens to EXEC Group for BUY/SELL commands, executes orders via Derayah REST API, and forwards results to Amin's DM.
EXEC Group	-5235925419	Private Telegram group used as the command bus between the poller (which sends order commands) and the execution bot (which receives and executes them). Also receives all trade alerts.
Derayah	Broker	Saudi brokerage. Order execution via REST API (api.derayah.com/trading). Portfolio ID: 2063853. Web platform: newonline.derayah.com. Trade terminal: derayah.tickerchart.net.
TickerChart	Data Feed	Real-time price feed embedded in Derayah Trade tab. Provides live WebSocket price stream (QO.SYMBOL.TAD topics) and hosts the JWT Bearer token used for API calls.

3. System Components

screener.py	<i>[Pre-market]</i>	Runs at 09:50 Sun-Thu. Scans 269 Sharia-compliant TASI stocks. Scores each by momentum, volume, and VWAP proximity. Outputs top 1-2 picks to picks.json. Classifies market regime (TRENDING/NEUTRAL/DEFENSIVE) via TASI index + oil price signals. Sends picks to Amin via Telegram.
poller.py	<i>[Market Hours]</i>	Runs 10:00-15:30 Sun-Thu. Every 5 min: fetches live prices via WS cache (TickerChart) or yfinance fallback. Checks entry signals (VWAP reclaim / breakout). Monitors open positions for stops. Auto-executes BUY/SELL via EXEC group. Exits at 15:30.
bot.py	<i>[Always On]</i>	Telegram bot running permanently. Receives commands from EXEC group and Amin directly. Executes orders via Derayah REST API. Runs 15-min keepalive loop: checks Derayah sessions, TC tab JWT, and REST API health. Forwards order results to Amin's DM.
derayah_keepalive.py	<i>[Every 5 min]</i>	Cron-driven (OpenClaw). Checks browser up, Derayah tab open, session not expired. Auto-starts Google Chrome if down. On session expiry, attempts login in priority order: (1) API refresh token injection - ~15s, no reCAPTCHA, no OTP; (2) CDP real Chrome login; (3) Playwright stealth login. Scrolls page to prevent idle timeout. Notifies Amin max 2x/day only if all three methods fail.
derayah_api.py	<i>[Library]</i>	Async REST client for api.derayah.com. Reads JWT Bearer token from TickerChart localStorage. Token cached 20 min, re-read from TC tab on expiry. Handles order placement, cancellation, portfolio/position queries.
market_regime.py	<i>[Library]</i>	Classifies TASI session as TRENDING / NEUTRAL / DEFENSIVE. Pre-market: uses TASI index 10d momentum + oil price signal. Intraday: updates every 30 min based on session % and VWAP position. Regime drives position sizing and max cycles.
map_selectors.py	<i>[10:05 daily]</i>	Runs at market open. Maps Derayah UI CSS selectors for symbol 1010. Ensures bot can find order fields if Derayah UI updates. Sends result to Amin via Telegram.
ws_probe.py	<i>[10:07 daily]</i>	Probes TickerChart WebSocket for 90s at market open. Validates that live price feed is flowing. Alerts Amin if no price frames received.
post_market_v2.py	<i>[Post-market]</i>	Runs at 15:35 Sun-Thu. Comprehensive post-market analysis with learning loop. Analyzes picks performance, finds missed opportunities, generates strategy recommendations. Posts report to TASI Execution group. Saves reports to /home/mino/tasi-exec/reports/, tracks learning progress in learning.json.
ws_logger.py	<i>[Utility]</i>	WebSocket price feed logger. Captures TickerChart WebSocket frames for debugging and audit.
start-chrome.sh	<i>[Utility]</i>	Bash script to start Google Chrome with CDP on port 18801, SOCKS5 proxy, single-process mode. Run after reboot or when browser is down.

4. Cron Schedule & Automation

All times are Riyadh (GMT+3). Trading days: Sunday-Thursday.

System Crontab (crontab -l)

Time	Job	Script	Purpose
Fri 03:00	Weekly reboot	sudo reboot	Clears memory, resets sessions
Sun-Thu 10:05	Selector mapper	map_selectors.py 1010	Maps Derayah UI for symbol 1010
Sun-Thu 10:07	WS probe	ws_probe.py 90	Validates live price feed (90s)

OpenClaw Agent Crons

Schedule	Name	Status	Purpose
Every 5 min	derayah-keepalive	ENABLED	Keep Chrome + Derayah tab live; auto-login on session expiry
Sun-Thu 09:50	tasi-premarket-screener	ENABLED	Run screener.py; send picks + regime to Amin
Sun-Thu 10:00	tasi-price-poller	ENABLED	Start poller.py; sends confirmation to Amin when live
Sun-Thu 15:35	post-market-analysis	ENABLED	Run post_market_v2.py; analyze picks, find missed opportunities, post report
Thu 22:00	sharia-list-refresh	ENABLED	Refresh Sharia-compliant stock list from Saudi Exchange
Every 30 min	ocean-health-monitor	ENABLED	System infrastructure: RAM/disk/CPU checks (not trading-specific)
Every 2 h	auth-token-sync	ENABLED	System infrastructure: Claude CLI token sync (not trading-specific)
Daily 04:00	daily-ram-cleanup	ENABLED	System infrastructure: RAM cleanup (not trading-specific)

Always-On Systemd User Services

socks-tunnel.service	autossh SOCKS5 tunnel to Amin-PC (port 1080) - Restart=always
openclaw-gateway.service	OpenClaw AI gateway (port 18789) - agent orchestration
chromium-derayah.service	Google Chrome with CDP on port 18801 - Derayah sessions
ocean-dashboard.service	System status dashboard (port 8765)

5. Daily Trading Operation Flow

Each trading day (Sunday-Thursday) follows this sequence automatically:

04:00	RAM Cleanup	Ocean self-cleans memory. Kills idle processes, restarts bloated services. Amin notified if action taken.
09:50	Pre-market Screener	screener.py scans 269 Sharia stocks. Classifies regime from TASI + oil. Writes top 1-2 picks to picks.json. Sends Telegram message to Amin: "Regime: DEFENSIVE Picks: 8030 (14.25-15.14 SL:13.90), 3060 (15.83-16.00 SL:14.92)"
10:00	Poller Start	tasi-price-poller cron starts poller.py. Sends: "Price poller live - fast watch every 10s, price scan every 5min."
10:05	Selector Mapper	map_selectors.py verifies Derayah UI structure for symbol 1010. Telegram alert if UI changed.
10:07	WS Probe	ws_probe.py confirms TickerChart WebSocket is streaming live prices. Alert if dead.
10:00-15:30	Active Monitoring	Poller checks picks every 5 min: 1. Fetch live price (WS cache first, yfinance fallback) 2. Check entry signal: VWAP reclaim or breakout above prior high 3. If signal + regime allows: auto-execute BUY 4. Monitor open positions: hard stop (-7%), trailing stop (activates at +2%), hard close at 14:45
14:45	Hard Close	All open positions force-sold via auto_sell(). Alert sent to Amin.
15:30	Session End	Poller exits. Sends "Price poller stopped (market closed)" to Amin.
After close	Post-Market Analysis	post_market_v2.py runs at 15:35. Analyzes all 269 Sharia stocks vs today's picks. Finds missed opportunities, generates strategy recommendations. Posts report to TASI Execution group. Saves report to reports/post_market_YYYY-MM-DD.html. Updates learning.json with sessions_analyzed, recommendations_made, missed_opportunities_avg.
After close	Keepalive	Derayah browser maintained but no new orders. Next screener at 09:50 next day.

6. Execution Flow - Order Lifecycle

Auto-Execution Path (no human intervention)

- 1. poller.py detects entry signal (VWAP reclaim / breakout)
- 2. Checks regime: DEFENSIVE=1 cycle 20%, NEUTRAL=2 cycles 40%, TRENDING=8 cycles 45%
- 3. Sends "BUY 1010 100 MARKET" to EXEC Group (Telegram)
- 4. Sends "Buying 1010 - order sent, confirmation incoming..." to Amin's DM
- 5. bot.py receives command from EXEC Group
- 6. bot.py reads JWT token from TickerChart localStorage (TC_DERAYAH)
- 7. bot.py calls POST api.derayah.com/trading/Order/Place
- 8. On success: bot.py replies "[OK] BUY 100 × 1010 @ Market - orderId=..." to EXEC Group + Amin DM
- 9. On failure: bot.py replies "[ERR] Order failed: <reason>" to EXEC Group + Amin DM
- 10. poller.py tracks position in positions.json

Manual Execution Path (Amin sends command)

- Amin sends "BUY 1010 100 MARKET" or "BUY 1010 100 @ 45.50" to EXEC Group
- bot.py validates: command must come from GROUP_CHAT_ID or OWNER_ID
- Execution proceeds same as steps 6-9 above
- Result returned as reply in EXEC Group

Token Source (Critical)

ALL orders require a valid JWT Bearer token. This token lives in the TickerChart tab's localStorage (key: TC_DERAYAH). The TickerChart tab (derayah.tickerchart.net) auto-refreshes it from the active Derayah session. bot.py caches the token for 20 minutes and re-reads on expiry. If the TC tab logs out, orders fail with HTTP 401.

TC Session Recovery (bot.py keepalive - every 15 min)

- bot.py reads JWT from TC localStorage, checks expiry
- If expiring within 5 min or missing: clicks 'Derayah Trade' link on newonline.derayah.com
- Waits up to 25s for TC tab to open and populate fresh JWT
- If TC tab shows login page: alerts Amin via EXEC Group
- If newonline.derayah.com session expired: attempts auto-login (Playwright, stored creds)
- If auto-login hits OTP/2FA: alerts Amin to complete manually

7. Risk Controls & Stop Logic

Position-Level Controls

- Hard Stop Loss: -7% from entry price -> immediate MARKET SELL
- Trailing Stop: activates after +2% gain, follows price up, exits on reversal
- Hard Close at 14:45: all open positions force-sold regardless of P&L
- Max Cycles per Pick: TRENDING=8, NEUTRAL=2, DEFENSIVE=1 (re-entries after stop-out)
- Scratch Guard: 2 consecutive scratches (small loss) stops cycling for that symbol today
- Entry Cutoff: no new BUY orders after 14:30

Regime-Based Position Sizing

Regime	Trigger	Position Size	Max Cycles	Strategy
TRENDING	TASI +momentum, oil positive, session above VWAP	45%	8	C
NEUTRAL	Mixed signals or default	40%	2	B
DEFENSIVE	TASI negative, oil down, or session below VWAP	20%	1	B

System-Level Controls

- SOCKS5 tunnel: all Derayah traffic routed via Amin-PC residential IP - prevents ISP blocks
- Browser health: derayah-keepalive cron detects browser crash, auto-restarts Chromium
- Session health: bot.py 15-min keepalive checks TC JWT, REST API ping, Derayah online session
- Order deduplication: positions.json tracks open positions - poller skips re-entry if already in
- Daily max login prompts: 2 per day - prevents Telegram spam if session repeatedly expires

8. Alert & Escalation Matrix

Alerts are sent via Telegram. Destination: Amin DM (5529987063) unless noted.

Alert	Message Sample	Destination	Response
Entry Signal	BUY 1010 100 MARKET - Cycle 1/1...	Amin DM + EXEC Group	Auto-execute or override manually
Order Confirmed	[OK] BUY 100 × 1010 @ Market - orderId=...	Amin DM	No action needed
Order Failed	[ERR] Order failed: 401 Unauthorized	Amin DM + EXEC Group	Check TC tab session, re-login if needed
Stop Loss Hit	[STOP] 8030: HARD STOP at -7% - selling	EXEC Group	Order auto-sent; confirm on Derayah
Hard Close 14:45	[ALERT] 14:45 HARD CLOSE - auto-selling.	EXEC Group	All positions force-closed
Session Expired	[WARN] Derayah session expired - please log...	Amin DM (max 2x/day)	Log in at newonline.derayah.com
TC Session Expired	[WARN] Derayah Trade session expired	EXEC Group	Log in to TickerChart tab
Browser Down	Browser failed to start - check Ocean [RED]	Amin DM	Check Ocean; restart Chromium
RAM Warning	[YELLOW] RAM Warning on Ocean - X% av	Amin DM	Close Chrome tabs if below 15%
RAM Critical	[RED] RAM: X% - Swap: XGB	Amin DM	Immediate: daily-cleanup or manual kill
Tunnel Down	SOCKS5 tunnel to Amin-PC is DOWN [WAR	Amin DM	Check Amin-PC is on and SSH accessible
Regime Change	Regime: NEUTRAL -> DEFENSIVE (reason)	Amin DM	Informational; affects future sizing
Screener Picks	Regime: X Picks: SYMBOL (zone, SL)...	Amin DM	Review before market open
Poller Live	Price poller live - fast watch 10s...	Amin DM	Confirm before market opens
Poller Stopped	Price poller stopped (market closed)	Amin DM	End of trading session
Sharia Refresh	[OK] Sharia list refreshed - N stocks	Amin DM	Every Thursday night
Token Expired	[WARN] Refresh token expired - manual logi	EXEC Group + Amin DM	Log in via CRD (PIN 056187); login_monitor.py c

9. Session Health & Keepalive Architecture

Two independent keepalive systems run in parallel to ensure Derayah sessions stay alive:

A) derayah_keepalive.py (cron, every 5 min, via OpenClaw)

- Checks Chromium is running on CDP port 18801 - starts it if not
- Finds newonline.derayah.com tab - opens it if missing
- Detects session expiry (URL = onboarding.derayah.com/##signin)
- LOGIN METHOD 1 (Primary): `_auto_login_token_inject()` - reads stored OAuth2 refresh token from `derayah_token_live.json`, calls Derayah IdentityServer (`api.derayah.com/idspark/connect/token`) to get a fresh access token, injects both tokens into browser localStorage, reloads page. No reCAPTCHA, no OTP, completes in ~15 seconds.
- LOGIN METHOD 2 (Fallback 1): `_auto_login_cdp()` - real Chromium via CDP, fills credentials with native JS setter events
- LOGIN METHOD 3 (Fallback 2): `_auto_login()` - Playwright stealth with stored creds, `press_sequentially` for Vue form
- Navigates to trading-portfolio and scrolls every 15 min to prevent idle timeout
- Alerts Amin max 2x/day only if all three login methods fail

B) bot.py keepalive loop (every 15 min, background thread)

- Checks newonline.derayah.com tab - auto-logins if session expired
- Reads TickerChart JWT from localStorage - checks expiry timestamp
- If JWT expiring < 5 min: clicks 'Derayah Trade' to re-open TC tab with fresh token
- Detects TC tab login page - alerts Amin via EXEC Group
- Pings REST API (`get_orders`) to keep JWT session alive server-side

OAuth2 Token Store (derayah_token_live.json)

The file `/home/mino/tasi-exec/derayah_token_live.json` stores the live Derayah OAuth2 tokens. Keys: `Derayah_accesstoken` (JWT, 60 min TTL) and `Derayah_refresh_token` (64-char hex, `offline_access` scope, valid 30+ days). Token endpoint: `https://api.derayah.com/idspark/connect/token`. Client: `NewWebClient`. Scope: `openid profile roles offline_access derayah.api`. Tokens are updated each time `_auto_login_token_inject()` or `_refresh_token_api()` runs. `login_monitor.py` (daemon) captures fresh tokens automatically after any manual login via CRD.

Browser Profile Persistence

Google Chrome uses `/home/mino/.config/google-chrome/derayah-profile/` as user data dir. Session cookies persist across browser restarts. On browser crash + restart, Derayah typically re-loads without requiring login (cookies valid ~24h). TC tab JWT expires independently and may require re-login even if cookies are valid.

10. Recovery Procedures

Browser Down

- Automatic: `derayah-keepalive` cron detects within 5 min, starts Google Chrome, attempts auto-login
- Manual via SSH: `/home/mino/tasi-exec/start-chrome.sh` (recommended) or `google-chrome-stable --remote-debugging-port=18801 --user-data-dir=/home/mino/.config/google-chrome/derayah-profile/ &`
- Manual via CRD: open Chrome Remote Desktop (PIN 056187), start Chrome manually

Derayah Session Expired

- Automatic (normal case): `derayah-keepalive` uses OAuth2 refresh token - no reCAPTCHA, no OTP, ~15 seconds
- Tokens stored in: `/home/mino/tasi-exec/derayah_token_live.json` (refresh token valid 30+ days)
- If refresh token expired (<1x per month): CDP or Playwright fallback auto-attempted
- If all auto-methods fail: log in manually via CRD (PIN 056187) at `newonline.derayah.com`
- After manual login: `login_monitor.py` daemon auto-captures new tokens to `derayah_token_live.json`
- To start capture daemon before manual login: `python3 /home/mino/tasi-exec/login_monitor.py`

TickerChart Session Expired (orders blocked)

- bot.py keepalive should auto-recover by re-opening TC tab (every 15 min)
- If not recovered: open CRD, navigate to newonline.derayah.com, click 'Derayah Trade'
- Verify token present: check bot.py log for 'TC tab: re-opened with fresh JWT'

Post-Market Analysis Not Running

- Check: verify post-market-analysis cron is enabled (15:35 Sun-Thu)
- Check: ls /home/mino/tasi-exec/reports/ for daily reports
- Manual run: python3 /home/mino/tasi-exec/post_market_v2.py
- Check learning.json: cat /home/mino/tasi-exec/learning.json

SOCKS5 Tunnel Down

- Automatic: socks-tunnel.service restarts every 15s via Restart=always
- Manual: systemctl --user restart socks-tunnel.service
- Root cause: Amin-PC offline or SSH port blocked - check PC is running

Ocean RAM Critical (<10% available)

- daily-ram-cleanup runs at 04:00 automatically
- Manual: pkill -u mino chromium; sleep 2; start browser fresh
- Check: ps aux --sort=-%mem | head -10
- Last resort: systemctl --user restart openclaw-gateway

Poller Not Running During Market Hours

- Automatic: tasi-price-poller cron starts it at 10:00
- Manual: cd /home/mino/tasi-exec && nohup python3 poller.py >> poller.log 2>&1 &
- Verify: tail -f /home/mino/tasi-exec/poller.log

11. Known Issues & Fixes Applied (2026-05-18) - 16 entries

Issue	Root Cause	Fix
CDP Port Mismatch	map_selectors.py and ws_probe.py used port 18800;	Updated CDP_URL to 18801 in both scripts
Duplicate Keepalive Runs	System crontab + OpenClaw cron both ran keepalive every 5 mins	Removed system crontab entry; OpenClaw is sole runner
Telegram 400 on Error Message	tg() sent raw stack traces with < > as HTML - Telegram	Added html.escape() on messages containing angle brackets
auth-token-sync Delivery	49 consecutive errors: delivery target 'last' with null chat_id	Added to explicit telegram:5529987063; set mode=none (silent)
D: Drive Monitor Orphaned	Recovery completed 2026-05-17; cron kept running e	Disabled cron a9dc595b
Auto-login Vue Form Bug	Playwright fill() didn't trigger Vue reactive validation; submit	Changed disabled sequentially() + wait_for(state=enabled)
WS Cache Miss - 3060.SR	Low-volume stock (818 trades/day); TC only sends or	on_frame now accepts bidprice/askprice as fallback mid-price; T
Poller Market Close 15:05	Poller hardcoded to exit 25 min before actual close (15:30)	Changed MARKET_CLOSE to time(15, 30)
yfinance Early Session Failure	period='1d' returns empty df in first 15 min of session	Fallback to period='5d' when 1d is empty
Order Confirmation Gap	auto_buy/sell sent 'Auto-bought' before order confirmed	Messages now say 'XFC Group' only; forwards [OK]/[ERR] to A
reCAPTCHA v2 Blocked All Au	Derayah login page uses reCAPTCHA v2 checkbox +	Bypassed entirely via OAuth2 refresh token API - no browser
Missing client_secret for Token	Refresh token endpoint returned 'invalid_client'; refres	Found client_secret in New Derayah Web 2.0 API; Derayah JS bund
OTP Screen Detection (SPA)	Derayah login is a Vue SPA - URL stays at onboardin	Fixed by checking DOM for OTP input field visibility instead of
login_monitor.py Timing Miss	Double-fork daemon launched to capture tokens after	Tokens no longer delayed by waiting 15s for token endpoint before
SOCKS5 Missing in requests	requests library raised 'Missing dependencies for SO	Removed proxy from token endpoint call - api.derayah.com token
Keepalive: Browser Context Closed	derayah_keepalive.py killed Google Chrome on sessio	Now fix, re-established; added failure logpoint (Sw) before log page; ca

NOTE: This document reflects the system state as of 2026-05-20. Changes to scripts, crons, or infrastructure should be logged in /home/mino/.openclaw/workspace/memory/issues.md and this document regenerated.